

计划 010

本地 Ollama LLM 替代 Hermes Docker

AMD Strix Halo 395 MAX 全栈优化

问题

当前 LLM 后端 = Hermes Docker (Linux 容器)

- 占用 ~1-2GB RAM (Linux VM)
- 需要 Docker Desktop 运行
- 增加启动延迟和系统复杂度

本机已有：Ollama 0.32.5 + meow36b (38GB) + qwen3.6:35b-a3b + ROCm 7.1

为什么 Strix Halo 独特

特性	普通 PC	AMD Strix Halo 395 MAX
GPU	独立显卡或无	Radeon 8060S (Vulkan + ROCm)
NPU	无	XDNA 2 NPU
内存	独立显存 + 系统内存	统一内存 ——iGPU 可访问 96GB 显存
LLM	需 CUDA 独立 GPU	Ollama + ROCm, 无需独立 GPU
STT	CPU 推理 (慢)	whisper.cpp Vulkan (GPU 加速)

独特价值：完全本地、零云端、零延迟预热的端到端 AI 语音代理

目标架构

```
浏览器 → QAA 网关 (:3101)
├── 在线语音 → DashScope 云端 (~0s)
├── 离线语音 → 本地 S2S (:8765)
│   ├── STT: whisper.cpp Vulkan (GPU, ~3s)
│   ├── LLM: Ollama qwen3.6:35b (GPU, ROCm)
│   └── TTS: Kokoro (CPU, 预热 ~0s)
└── 工具调用 → ACP shim → Ollama (:11434)
```

无 Docker · 无云端 API · 全部本地 GPU 推理

GPU 资源分配

AMD Strix Halo (iGPU 96GB 显存)

GPU (Radeon 8060S)

- └─ Ollama qwen3.6:35b LLM (38GB)
- └─ whisper.cpp STT (Vulkan)

CPU (16 核)

- └─ Kokoro TTS
- └─ VAD (Silero)
- └─ QAA 网关 (Node.js)

NPU (XDNA 2)

- └─ 未来: NPU STT (待 2026 末)

步骤概要

1. 确认 Ollama GPU — `ollama ps` 验证 ROCm 加速
2. 创建 agent 配置 — `examples/ollama-llm/config.yaml`
3. 启动脚本 — `scripts/start-ollama-llm.ps1` + 预加载模型
4. 更新启动栈 — `start-voice-stack.ps1` 支持 Ollama 模式
5. ACP shim 扩展 — `LLM_BASE_URL` 可指向 Ollama
6. 冒烟测试 — 语音 + 工具调用, 无 Docker

Hermes Docker vs Ollama 本地

指标	Hermes Docker	Ollama 本地
内存	~1-2GB (VM)	~0 (原生)
启动	~5-10s	~3s
推理	远程 API	GPU 本地
延迟	~1.8ms	~0.3ms
模型	固定	可切换
成本	API 费用	零成本

预期效果

组件	当前	优化后
LLM 预热	~5-10s (Docker)	~3-5s (GPU 加载)
LLM 推理	远程 API	本地 GPU (ROCm)
总启动	Docker + S2S + QAA	Ollama + S2S + QAA
云端依赖	需要 (API)	零 (完全本地)
成本	API 费用	免费